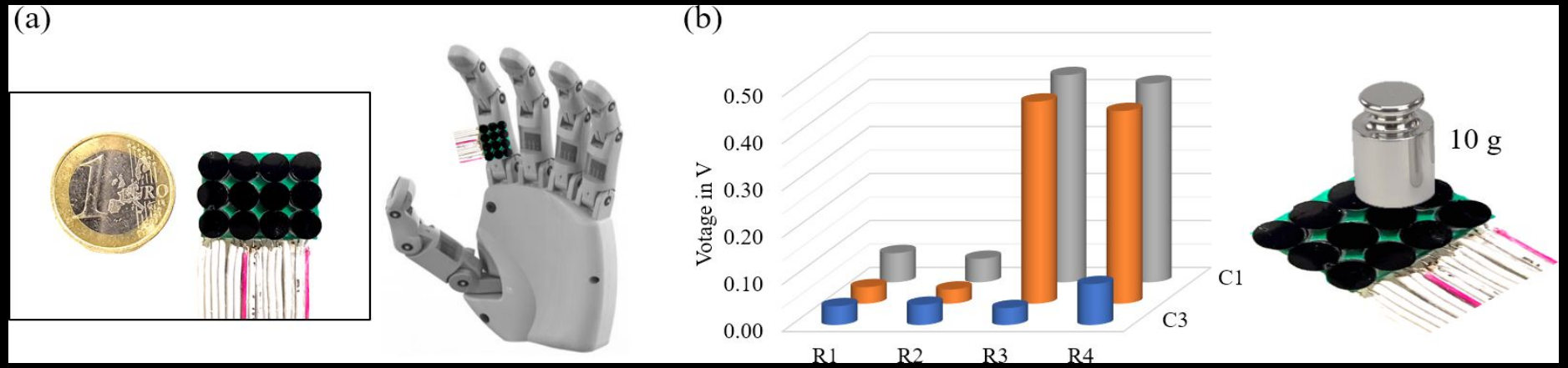
Analog Inputs in Robotics

By: Bilal Ahmed

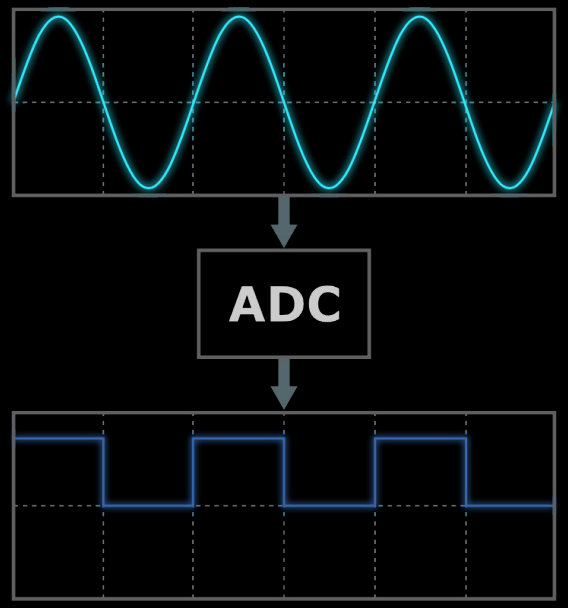
Why Robots Need Analog Sensing

- A robot's job is to react to the **physical world** — and the physical world is analog: light intensity, distance, force, temperature, tilt angle all vary smoothly, never just "on" or "off."
- A robot that could only sense HIGH/LOW would be blind to *how much* — it could tell "obstacle present," but never "how close," "how bright," or "how hard."
- **Analog input is what gives a robot a sense of degree, not just presence** — the foundation of any robot that needs to react proportionally rather than just react at all.



The ADC — Bridging Analog World and Digital Brain

- **ADC (Analog-to-Digital Converter)** samples a voltage and converts it into a number the microcontroller's CPU can compute with.
- Arduino Uno: **10-bit ADC** → 1024 levels (0–1023), mapped across 0–5V.
- Digital Value = (Voltage / 5V) × 1023
- Every analog sensor on a robot — distance, light, force — ultimately produces a **voltage**, and the ADC is the translator between that voltage and robot decision-making logic.

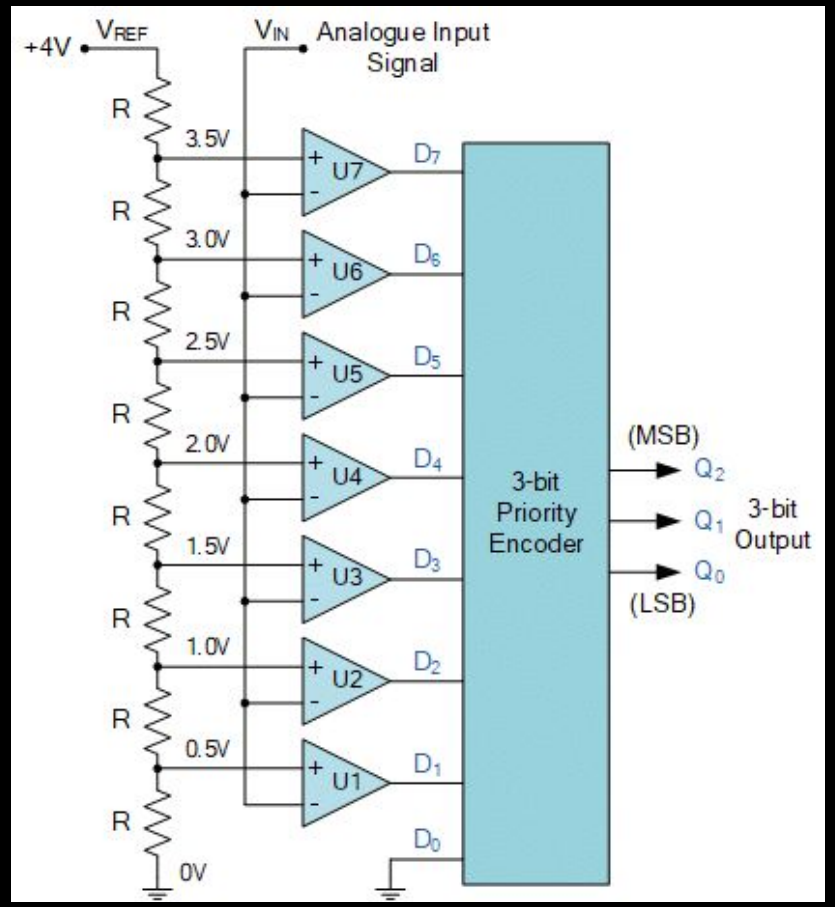


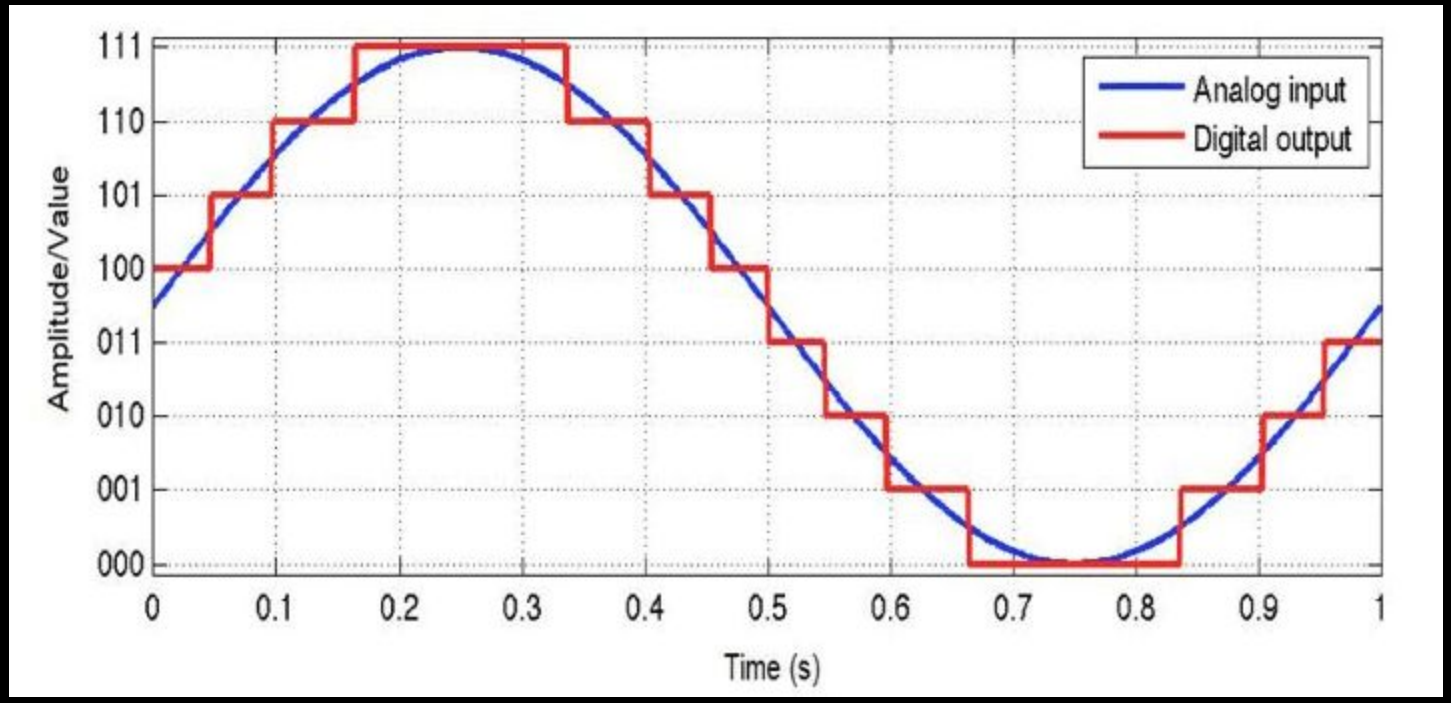
Simple Example: 3-Bit ADC Mapping – Binary Digital Output (0–6V)

- **ADC resolution:** 3-bit → outputs a **3-bit binary number** (000 to 111) = 8 possible digital signals
- **Reference voltage:** 6V
- Resolution = 6V / 8 = 0.75V per step

Voltage Range	Decimal Value	Binary Digital Signal
0.00V – 0.749V	0	000
0.75V – 1.499V	1	001
1.50V – 2.249V	2	010
2.25V – 2.999V	3	011
3.00V – 3.749V	4	100
3.75V – 4.499V	5	101
4.50V – 5.249V	6	110
5.25V – 6.00V	7	111

Let’s see its working





Arduino's actual ADC uses 10 bits, not 3. How many possible binary digital signals can it produce, and why does that matter for detecting small differences in a sensor reading?

Hint: You have already seen the answer.

Resolution – How Fine Can the Robot "Feel"?

- **Resolution** = smallest voltage difference the ADC can distinguish.
- Resolution = 5V / 1024 ≈ 4.9 mV per step
- **Robotics relevance:** A robotic gripper using a force sensor with low resolution might not be able to tell the difference between "holding an egg gently" and "about to crush it“
- Resolution directly limits how *fine* a robot's sense of touch or distance can be.
- Higher-end robots often use 12-bit or 16-bit ADCs (4096–65536 levels) specifically where fine control matters — e.g., precision surgical robots reading force feedback.

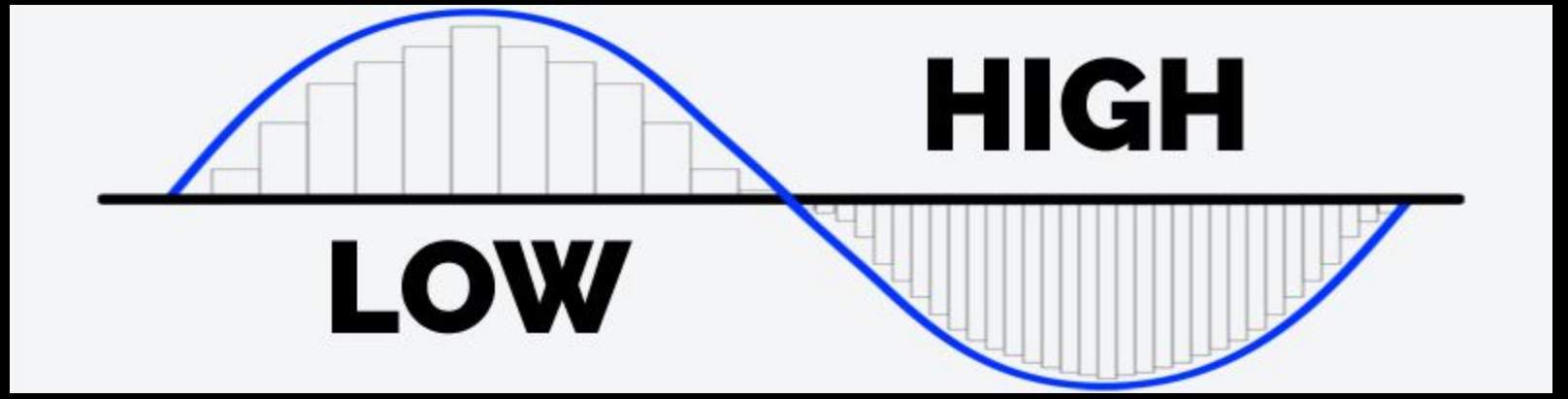


Sampling – Capturing a Signal That's Always Changing

- **Sampling** = taking a snapshot reading at one instant; **sampling rate** = how often this happens per second.
- A robot moving fast needs to sample its sensors fast too — a slow sampling rate means the robot reacts to "old," outdated information.
- **Robotics relevance:** A self-balancing robot (like a segway-style bot) samples its tilt sensor hundreds of times per second
- Sample too slowly, and the robot physically falls over before it "notices" it's tipping.



Sampling Rate Trade-Off – Low vs. High



Benchmark	Low Sampling Rate	High Sampling Rate
Signal accuracy	Misses fast changes — signal appears jagged/stepped or incomplete	Closely tracks the real signal, smooth and accurate
Data volume	Small — fewer readings to store/process	Large — much more data generated per second
CPU/processing load	Low — plenty of time between samples for other tasks	High — CPU spends more time sampling, less on other logic
Power consumption	Lower — fewer ADC operations	Higher — constant sensor polling drains battery faster
Response speed to changes	Slow — robot may "react late" to fast events	Fast — robot detects and responds almost immediately
Risk of missing critical events	High — a brief spike/dip between samples goes unnoticed	Low — brief events are still captured

What is the idea sampling rate?

Should we do the highest always?

No universally "correct" sampling rate

It depends entirely on how fast the real-world signal changes, and how fast the robot needs to react.

Fast-changing physical processes (balance, motor feedback, obstacle distance at high speed) → **need high sampling rate**.

Slow-changing processes (ambient temperature, battery level over time) → **low sampling rate is sufficient, and more efficient**.

A drone's gyroscope samples thousands of times per second, but its GPS module might only update once per second. Why is that difference intentional, not a limitation?



This Photo by Unknown Author is licensed under CC BY-NC-ND

Reading a Real Analog Sensor — Code

```
int sensorPin = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int rawValue = analogRead(sensorPin);   // 0–1023
  Serial.println(rawValue);
  delay(50);                              // sampling interval
}
```

This single pattern — analogRead() → interpret → decide — is reused across nearly every analog-sensing robot, regardless of what the sensor actually measures.

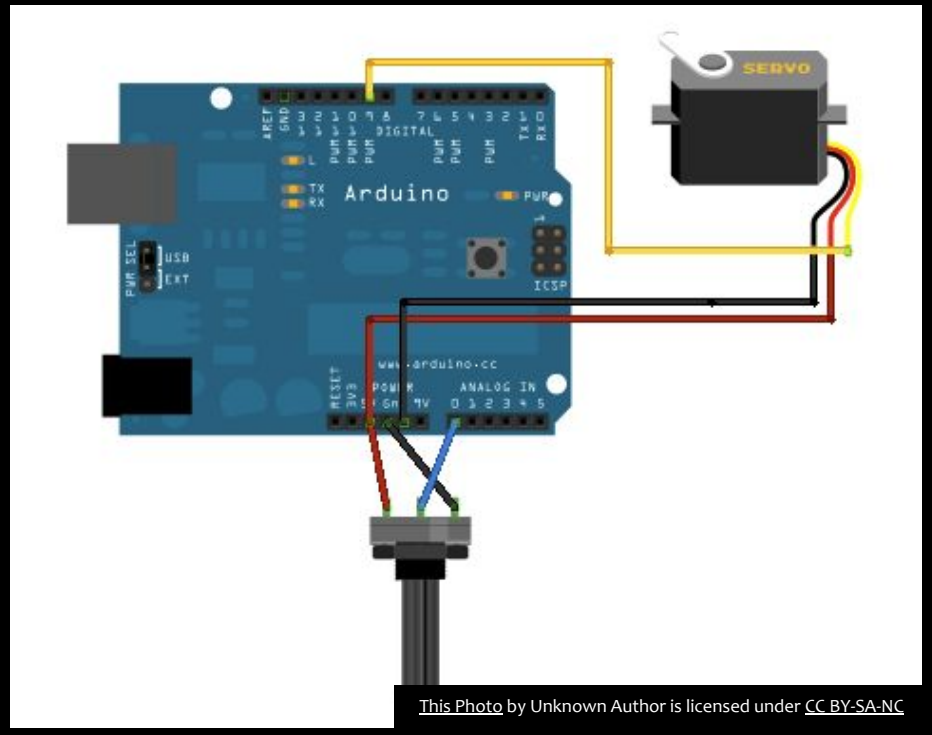
Famous Robots That Rely on Analog Sensing

Robot	Analog Input Used	Purpose
NASA Mars Rovers (Curiosity, Perseverance)	Analog sensors for temperature, tilt, sun angle	Environmental monitoring, safe navigation on uneven terrain
iRobot Roomba	IR/analog cliff sensors (light-based)	Detecting stairs/edges to avoid falling
Boston Dynamics' Spot	Analog force/torque sensors in joints	Balancing, adapting leg force on uneven ground
DARPA Grand Challenge autonomous cars (early self-driving research vehicles)	Analog inputs from early distance/proximity sensors alongside LiDAR	Obstacle distance estimation before full sensor fusion
Line-following robots (common university/competition robots)	LDR / IR reflectance sensors (analog)	Detecting line position by reading reflected light intensity

Analog Devices in Arduino

Potentiometer

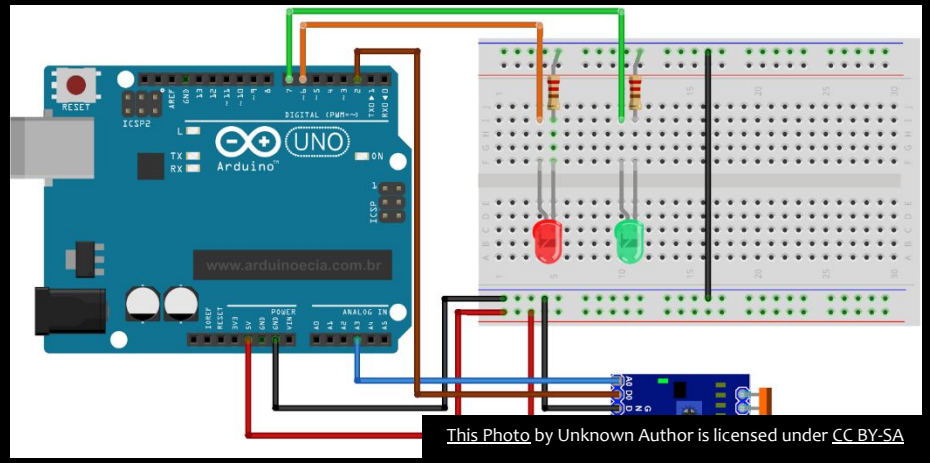
- A **variable resistor** with a rotating/sliding wiper — acts as a manual analog voltage divider.
- Output voltage changes **linearly** as the knob turns, from 0V to 5V.
- **Wiring:** Two outer legs → 5V and GND; middle wiper → Arduino analog pin (A0).
- **Robotics Application:** Manual speed/angle control — joystick-style input for robotic arms, manual override for motor speed, tuning sensor thresholds live during testing.



This Photo by Unknown Author is licensed under CC BY-SA-NC

LDR (Light Dependent Resistor)

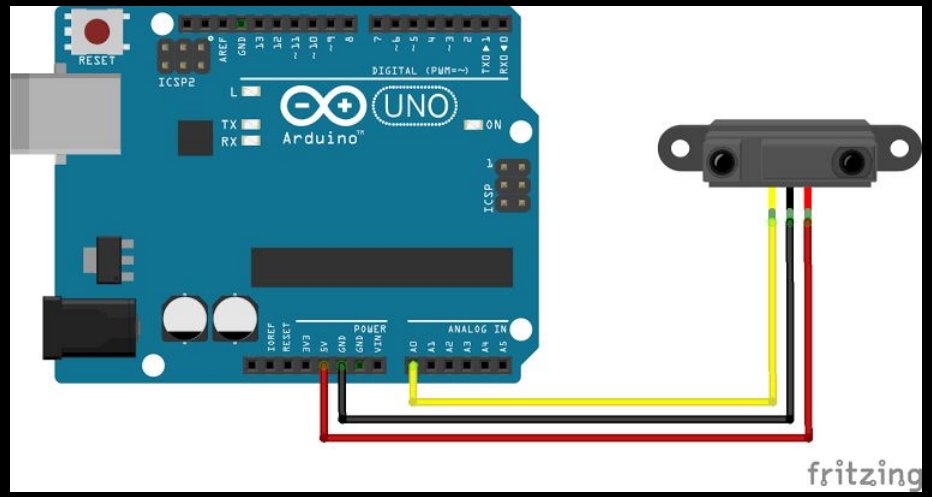
- Resistance **decreases as light increases** — used in a voltage divider circuit with a fixed resistor to produce a light-dependent analog voltage.
- **Wiring:** LDR + fixed resistor (e.g., 10kΩ) in series between 5V and GND; midpoint connects to an analog pin.
- **Robotics Application:** Light-seeking/avoiding robots, automatic headlight activation on mobile robots, line-following robots (detecting reflected light off a dark line vs. light surface).



This Photo by Unknown Author is licensed under CC BY-SA

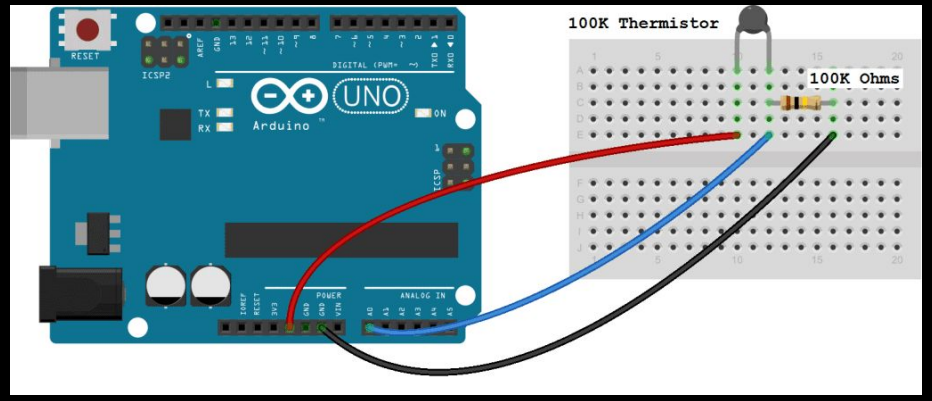
Sharp IR Distance Sensor

- Infrared emitter + receiver measures distance via reflection angle — outputs an **analog voltage inversely proportional to distance**.
- Non-linear relationship (voltage vs. distance follows a curve, not a straight line).
- **Wiring:** 3 pins — VCC, GND, and Vo (analog output) → Arduino analog pin.
- **Robotics Application:** Obstacle avoidance, wall-following in maze-solving robots, short-range collision prevention on autonomous mobile robots.



Thermistor (Temperature Sensor)

- Resistance **changes with temperature** — used in a voltage divider (like the LDR) to produce a temperature-dependent analog voltage.
- Commonly paired with a fixed resistor; voltage-to-temperature conversion
- **Robotics Application:** Overheat protection for motors/batteries in mobile robots, environmental monitoring robots, temperature-based automation in agricultural or industrial robotics.



Soil Moisture Sensor

- Two probes measure the **electrical conductivity of soil** — moist soil conducts more current (lower resistance), dry soil conducts less (higher resistance).
- **Wiring:** VCC, GND, and AO (analog output) → Arduino analog pin.
- **Robotics Application:** Autonomous agricultural robots for irrigation decisions, smart farming rovers that water only where soil is dry, environmental monitoring robots mapping soil conditions across a field.

